

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN – TRƯỜNG CÔNG NGHỆ
KHOA KHOA HỌC DỮ LIỆU VÀ TRÍ TUỆ NHÂN TẠO

CẨM NANG HỌC PHẦN & TỰ HỌC PYTHON

DSAI1003: CÁC NGUYÊN LÝ LẬP TRÌNH CƠ BẢN VỚI PYTHON

Fundamental Programming Concepts in Python

Giảng viên: TS. Vũ Đức Minh
Chương trình:

Mã học phần: DSAI1003 **Khối lượng:**

Quyết định: 975/QĐ-ĐHKTQD
(30/03/2024)

Website: dmvn.github.io/Fundamental-of-Programming-Concepts-in-Python

Khoa học dữ liệu (DSFE)
3 Tín chỉ (45h LT + 22.5h TH + 90h tự học)
Bắt buộc cơ sở ngành

Contents

1	Tổng quan Học phần & Đề cương chi tiết (Syllabus)	1
1.1	Giới thiệu học phần & Triết lý tiếp cận	1
1.2	Mục tiêu học phần (COs) & Chuẩn đầu ra (CLOs)	2
1.3	Giáo trình & Tài liệu học tập	2
1.4	Phương pháp đánh giá & Quy định học phần	2
1.5	Kế hoạch giảng dạy chi tiết 15 tuần	3
2	Phương pháp Tự học Python Hiệu quả	4
2.1	Bản chất của học lập trình & 4 ngộ nhận tai hại	4

2.2	Chu trình tự học 9 bước (The 9-Step Self-Study Cycle)	4
2.3	Các kỹ thuật học tập chủ động đột phá	4
2.3.1	Kỹ thuật Predict → Run → Explain (Dự đoán → Chạy → Diễn giải)	4
2.3.2	Kỹ thuật Teach-Back (Giảng ngược lại)	5
2.3.3	Kỹ thuật Active Recall & Spaced Repetition (Lặp lại ngắt quãng)	5
2.4	Nấc thang luyện tập 5 cấp độ (Learning Ladder)	5
2.5	Quy trình gỡ lỗi (Debugging Checklist) & Phân loại lỗi	6
2.6	Xây dựng Nhật ký học tập (Learning Log) & Error Log	6
3	Chiến lược Sử dụng Trí tuệ Nhân tạo (AI) Hỗ trợ Học tập	7
3.1	Định vị vai trò chuẩn xác của AI trong học lập trình	7
3.2	Khung cấu trúc Prompt 4 thành phần	7
3.3	Nguyên tắc 5T khi học cùng AI	7
3.4	Hệ thống gợi ý 3 cấp độ (Three-Level Hint System)	8
3.5	Bộ công cụ Prompt thực chiến (Prompt Toolkit)	8
3.6	Bảng kiểm tra định thông tin do AI sinh ra (Verification Checklist)	9
4	Kế hoạch Hành động Khởi đầu Học phần	10

1. Tổng quan Học phần & Đề cương chi tiết (Syllabus)

1.1 Giới thiệu học phần & Triết lý tiếp cận

Học phần **Các nguyên lý lập trình cơ bản với Python (DSAI1003)** là môn học nhập môn tiên quyết trong chương trình đào tạo Cử nhân ngành *Khoa học dữ liệu trong Tài chính và Thương mại điện tử (DSFE)*. Môn học được thiết kế nhằm xây dựng nền móng vững chắc về tư duy thuật toán, cấu trúc chương trình và kỹ năng giải quyết bài toán thông qua ngôn ngữ Python.

Đặc điểm tiếp cận môn học

- **Không đòi hỏi kinh nghiệm lập trình từ trước:** Sinh viên chỉ cần nắm chắc kiến thức đại số phổ thông và tư duy logic cơ bản.
- **Định hướng ứng dụng thực tế:** Các ví dụ, bài tập thực hành và case study được lấy trực tiếp từ lĩnh vực kinh tế, tài chính, phân tích dữ liệu bán lẻ và thương mại điện tử.
- **Lộ trình bài bản:** Xuất phát từ tư duy thuật toán (mã giả, lưu đồ), làm chủ các cấu trúc điều khiển cơ sở, tổ chức module bằng hàm, khai thác các cấu trúc dữ liệu tích hợp mạnh mẽ, cho đến nền tảng Lập trình hướng đối tượng (OOP).

1.2 Mục tiêu học phần (COs) & Chuẩn đầu ra (CLOs)

Môn học hướng đến 7 mục tiêu lớn (**G1 đến G7**), tương ứng với các chuẩn đầu ra đo lường được:

1.3 Giáo trình & Tài liệu học tập

Môn học sử dụng 3 bộ giáo trình chuẩn quốc tế:

1. **[1] Python for Everyone** (3rd Edition, 2019) – *Cay Horstmann, Rance Necaise*, John Wiley & Sons. (Giáo trình chính về cú pháp, thuật toán và bài toán kinh doanh).
2. **[2] Intro to Python for Computer Science and Data Science: Learning to Program with AI, Big Data and The Cloud** (2020) – *Paul Deitel, Harvey Deitel*, Pearson Education. (Giáo trình chính về tư duy xử lý dữ liệu và AI).

Mục tiêu	Mô tả Mục tiêu học phần (Course Objectives)	PLO	Mức độ
G1	Cài đặt chương trình console hoặc giao diện cơ bản giải quyết bài toán kinh doanh.	1.4, 2.2	4
G2	Vận dụng cấu trúc rẽ nhánh (if-elif-else) và vòng lặp (while, for) cho các tác vụ lặp.	1.4, 2.2	4
G3	Thiết kế và cài đặt hàm module hóa và hàm đệ quy chia nhỏ bài toán phức tạp.	1.4, 2.2	4
G4	Khai thác thành thạo các cấu trúc dữ liệu tích hợp sẵn (List, Tuple, Set, Dictionary).	1.4, 2.2	4
G5	Thao tác đọc/ghi tệp tin văn bản, tệp nhị phân và xử lý ngoại lệ an toàn (try-except).	1.4, 2.2	4
G6	Nắm vững và vận dụng kiến thức Lập trình hướng đối tượng (OOP: Lớp, Kế thừa, Đa hình).	1.4, 2.2	3
G7	Phát triển năng lực tự học, giải quyết vấn đề độc lập, làm việc nhóm và tinh thần trách nhiệm.	2.4, 3.1	3

Table 1: Bảng Mục tiêu học phần DSAI1003

3. [3] **The Python Workbook: A Brief Introduction with Exercises and Solutions** (2nd Edition, 2019) – Ben Stephenson, Springer. (Tài liệu bài tập tự luyện và đối chiếu lời giải).

1.4 Phương pháp đánh giá & Quy định học phần

Kết quả học tập được đánh giá qua 3 đầu điểm độc lập, bảo đảm tính liên tục và phản ánh chính xác năng lực thực chiến của sinh viên:

Hình thức đánh giá	Nội dung đánh giá	Thời điểm	Tiêu chí & Công cụ	Tỷ lệ
Bài tập về nhà (HW)	Lý thuyết & Bài tập lập trình thực hành	2 tuần/lần (≥ 4 bài)	Mức độ hiểu bài, kỹ năng giải thuật, đúng deadline (LMS/Moodle)	30%
Thi giữa kỳ (Midterm)	Kiểm tra lý thuyết & thuật toán (Bài 1 – 6)	Tuần 7	Tự luận trên giấy hoặc máy tính phòng Lab	30%
Thi cuối kỳ (Final)	Bài thi thực hành lập trình tổng hợp (Bài 1 – 10)	Tuần 15	Thi máy tính phòng Lab (chạy mã nguồn thực tế, OOP, bắt lỗi)	40%

Table 2: Thang đánh giá học phần DSAI1003

Quy định học phần cần lưu ý

- **Chuyên cần:** Sinh viên bắt buộc tham dự tối thiểu **80%** số buổi học trên lớp.
- **Điều kiện đạt môn:** Điểm tổng kết học phần phải đạt tối thiểu từ **5.0 / 10.0** trở lên.
- **Thiết bị:** Máy tính cá nhân (Laptop) chỉ được sử dụng cho mục đích thực hành lập trình theo yêu cầu của giảng viên; nghiêm cấm sử dụng thiết bị cho các việc riêng không liên quan.

1.5 Kế hoạch giảng dạy chi tiết 15 tuần

Lộ trình 15 tuần học được chia làm 2 giai đoạn logic và rõ ràng:

Giai đoạn 1: **Nền tảng cú pháp, cấu trúc điều khiển & Thuật toán (Tuần 1 – 7)**

Giai đoạn 2: **Dữ liệu nâng cao, Tệp & Lập trình hướng đối tượng OOP (Tuần 8 – 15)**

Tuần	Chủ đề & Nội dung trọng tâm	Tài liệu đọc	Hoạt động & Đánh giá
1	Bài 1: Giới thiệu chung & Thuật toán Máy tính, Python, lỗi biên dịch vs. runtime, mã giả, lưu đồ.	[1] Ch. 1 [2] Ch. 1	Giới thiệu đề cương, cài đặt môi trường (Python, VS Code, Colab).
2	Bài 2: Quy trình PM & Kiểu dữ liệu Số, chuỗi, biểu thức số học, lệnh gán, biến, hàm tích hợp.	[1] Ch. 2 [2] Ch. 2, 4	Live coding, tương tác Q&A. Giao Bài tập về nhà #1 .
3	Bài 3: Cấu trúc ra quyết định Câu lệnh if, if-else, if-elif-else, toán tử Boolean.	[1] Ch. 3 [2] Ch. 3	Thực hành rẽ nhánh, kiểm thử nhánh code. Thu bài tập #1.
4	Bài 4: Cấu trúc vòng lặp Vòng lặp while, for, lặp lồng nhau, duyệt chuỗi.	[1] Ch. 4 [2] Ch. 3	Luyện mẫu lặp (accumulator, sentinel). Giao Bài tập #2 .
5	Bài 5: Thiết kế hàm (Functions) Cú pháp def, tham số, giá trị trả về, phạm vi biến, đệ quy.	[1] Ch. 5 [2] Ch. 4	Module hóa, chia để trị. Thu bài tập #2.
6	Bài 6: Cấu trúc List và Tuple Mảng tuần tự, indexing âm, slicing, phương thức List, ma trận 2D.	[1] Ch. 6 [2] Ch. 5	Thao tác cấu trúc dữ liệu bằng. Giao Bài tập #3 .
7	Thi giữa kỳ (Midterm Exam) Ôn tập củng cố & Đánh giá kiến thức Bài 1 đến Bài 6.	[1] Bài 1–6	Tổ chức thi giữa kỳ (Trọng số 30%).

Table 3: Kế hoạch học tập Giai đoạn 1 (Tuần 1 – 7)

Tuần	Chủ đề & Nội dung trọng tâm	Tài liệu đọc	Hoạt động & Đánh giá
8–9	Bài 7: Thao tác Tập & Ngoại lệ Đọc/ghi tệp văn bản, tham số sys.argv, an toàn ngoại lệ (try-except).	[1] Ch. 7 [2] Ch. 4	Lưu trữ bền vững & xử lý lỗi phòng thủ. Giao Bài tập #4 .
10	Bài 8: Tập hợp (Set) & Từ điển (Dict) Bảng băm, tra cứu key-value, phép toán tập hợp, dữ liệu lồng nhau.	[1] Ch. 8 [2] Ch. 6	Xử lý dữ liệu giao dịch thương mại & phân tích kinh doanh.
11–12	Bài 9: Đối tượng & Lớp (OOP – Phần 1) Tư duy OOP, khai báo class, hàm tạo __init__, đóng gói, phương thức.	[1] Ch. 9 [2] Ch. 10	Mô hình hóa bài toán ngân hàng, hóa đơn. Mini-lab OOP.
13–14	Bài 10: Kế thừa & Đa hình (OOP – Phần 2) Cây kế thừa, lớp con, gọi super(), ghi đè method, đa hình.	[1] Ch. 10 [2] Ch. 10	Kiến trúc OOP nâng cao. Hướng dẫn ôn tập cuối kỳ.
15	Tổng kết học phần & Ôn thi cuối kỳ Tổng kết toàn bộ học phần DSAI1003, giải đáp thắc mắc sinh viên.	Toàn bộ tài liệu	Mock test trên máy tính & củng cố kỹ năng.

Table 4: Kế hoạch học tập Giai đoạn 2 (Tuần 8 – 15)

2. Phương pháp Tự học Python Hiệu quả

2.1 Bản chất của học lập trình & 4 ngộ nhận tai hại

Học lập trình không giống việc học thuộc lòng các môn lý thuyết, cũng không đơn thuần là ghi nhớ cú pháp lệnh. **Bản chất của lập trình là tư duy giải quyết vấn đề (problem-solving) và chuyển hóa thuật toán thành chỉ thị máy tính.**

Nguyên tắc tự học cốt lõi

"Học lập trình bằng cách đọc ít hơn, thực hành nhiều hơn và tự suy nghĩ trước khi xem lời giải."

Người mới học lập trình thường rơi vào 4 cái bẫy tâm lý sau:

1. **Ảo tưởng hiểu bài qua việc đọc lướt (Reading without running):** Đọc một đoạn code và tự nhủ *"mình hiểu rồi"*. Thực tế khi tắt tài liệu đi, sinh viên hoàn toàn không gõ lại được chương trình.
2. **Sao chép code mù quáng (Copy-pasting):** Chương trình chạy đúng nhưng người học không thể giải thích tại sao câu lệnh đó được dùng, tham số đó có ý nghĩa gì.
3. **Mở xem đáp án quá sớm:** Vừa gặp bài toán khó là lập tức tìm lời giải trên mạng hoặc hỏi AI. Hành động này triệt tiêu hoàn toàn quá trình rèn luyện cơ bắp não bộ về tư duy logic.
4. **Học dồn quá nhiều khái niệm trong một buổi:** Cố gắng học một mạch từ Biến → Rẽ nhánh → Vòng lặp → Hàm → OOP trong 2 ngày mà không có thời gian thẩm thấu qua bài tập.

2.2 Chu trình tự học 9 bước (The 9-Step Self-Study Cycle)

Để làm chủ một khái niệm lập trình mới, sinh viên được khuyến nghị tuân theo chu trình khép kín sau:

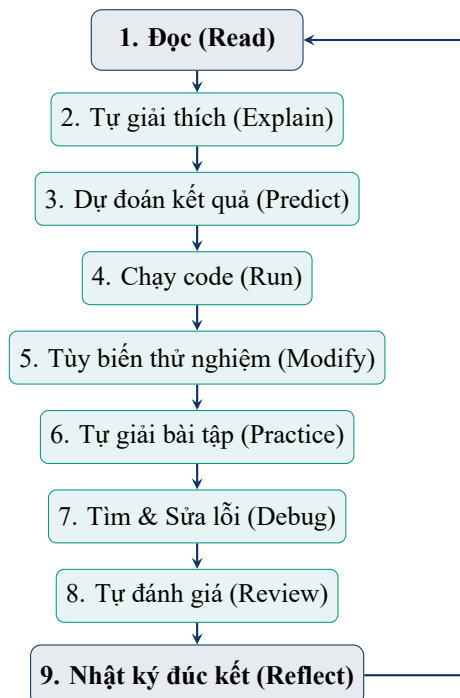


Figure 1: Chu trình tự học lập trình chủ động 9 bước

Thời gian tự học không nên chia đều cho các bước. **Hơn 70% thời gian phải dành cho các bước: Viết code, chạy thử, phân tích lỗi và giải quyết bài tập.**

2.3 Các kỹ thuật học tập chủ động đột phá

2.3.1 Kỹ thuật Predict → Run → Explain (Dự đoán → Chạy → Diễn giải)

Trước khi bấm nút Run trên IDE, hãy bắt buộc bản thân viết ra giấy hoặc nhẩm trong đầu:

- Chương trình này sẽ in ra màn hình chính xác những gì?
- Từng dòng lệnh thay đổi giá trị của biến số trong bộ nhớ như thế nào?

Nếu kết quả thực tế sai khác với dự đoán, đó chính là cơ hội vàng để bạn phát hiện ra "lỗ hổng" trong mô hình tư duy của mình về cách Python hoạt động.

2.3.2 Kỹ thuật Teach-Back (Giảng ngược lại)

Hãy thử giải thích một khái niệm (ví dụ: *Hàm trả về giá trị khác gì hàm in ra màn hình?*) cho một người chưa biết lập trình hoặc tự ghi âm lại lời giải thích của mình. Nếu bạn chỉ có thể nói: *"Nó chạy như thế vì nó phải thế"*, chứng tỏ bạn chưa thực sự hiểu sâu bản chất vấn đề.

2.3.3 Kỹ thuật Active Recall & Spaced Repetition (Lặp lại ngắt quãng)

Đừng ôn tập bằng cách đọc lại slide nhiều lần. Hãy đóng tài liệu lại và tự trả lời các câu hỏi: *Hàm input() trả về kiểu dữ liệu gì? Muốn đọc số nguyên thì phải ép kiểu ra sao? Nếu người dùng nhập chữ thì lỗi gì xảy ra?* Lên lịch ôn lại sau 1 ngày, 3 ngày, 7 ngày và 14 ngày để biến kiến thức ngắn hạn thành phần xạ dài hạn.

2.4 Nấc thang luyện tập 5 cấp độ (Learning Ladder)

Khi tiếp cận bất kỳ chủ đề nào, hãy đi theo từng nấc thang độ khó tăng dần:

- **Cấp độ 1 – Tùy biến ví dụ có sẵn:** Đổi giá trị biến số, đổi kiểu dữ liệu từ số nguyên sang số thực, quan sát kết quả thay đổi.
- **Cấp độ 2 – Viết lại từ mô tả bài toán:** Che lời giải mẫu, đọc đề bài và tự mình gõ lại toàn bộ chương trình từ con số không.
- **Cấp độ 3 – Kết hợp nhiều khái niệm:** Ví dụ: Viết chương trình vừa dùng vòng lặp for, vừa dùng nhánh if-else để lọc số chẵn lẻ trong danh sách.
- **Cấp độ 4 – Bài toán ngữ cảnh thực tế:** Tính tiền điện lũy tiến, tính thuế thu nhập cá nhân, tính lãi suất kép ngân hàng.
- **Cấp độ 5 – Mini Project:** Xây dựng ứng dụng quản lý giỏ hàng console, ứng dụng phân tích tệp dữ liệu chi tiêu cá nhân.

2.5 Quy trình gỡ lỗi (Debugging Checklist) & Phân loại lỗi

Gặp lỗi khi lập trình là điều tất yếu và là một phần bắt buộc của việc học. Khi chương trình báo lỗi, tuyệt đối không hoảng loạn hoặc xóa hết đi làm lại. Hãy áp dụng bảng kiểm tra sau:

Quy trình 8 bước Debugging tiêu chuẩn

1. Đọc toàn bộ thông báo lỗi từ dòng cuối cùng ngược lên (Traceback).
2. Xác định chính xác số dòng (Line number) gây ra lỗi.
3. Nhận diện loại lỗi: **SyntaxError** (sai cú pháp), **NameError** (biến chưa khai báo), **TypeError** (sai kiểu dữ liệu), **IndexError** (vượt chỉ số), **ZeroDivisionError** (chia cho 0).
4. Sử dụng các lệnh `print()` hoặc Debugger để in ra giá trị hiện thời của các biến.
5. Kiểm tra kiểu dữ liệu của biến bằng hàm `type()`.
6. So sánh giá trị thực tế nhận được với giá trị kỳ vọng trên lý thuyết.
7. Thử nghiệm với một bộ dữ liệu đầu vào nhỏ nhất, đơn giản nhất.
8. Chỉ sửa duy nhất **một yếu tố** trong mỗi lần thử, tránh sửa lung tung nhiều chỗ cùng lúc.

2.6 Xây dựng Nhật ký học tập (Learning Log) & Error Log

Sau mỗi buổi tự học, hãy dành 5 phút ghi lại nhật ký học tập (dạng tệp `learning_log.md`):

Lỗi gặp phải	Nguyên nhân gốc rễ	Cách khắc phục	Đã hiểu?
<code>TypeError: can only concatenate str to str</code>	Hàm <code>input()</code> luôn trả về chuỗi kí tự (<code>str</code>), không thể cộng trực tiếp với số nguyên.	Ép kiểu bằng <code>int(input())</code> khi tính toán.	Có
<code>IndexError: list index out of range</code>	Vòng lặp duyệt chỉ số từ 0 đến N , trong khi phần tử cuối cùng là $N - 1$.	Điều chỉnh giới hạn <code>range(len(lst))</code> .	Có
Vòng lặp chạy vô tận (Infinite loop)	Quên không cập nhật biến đếm bước lặp ở cuối thân vòng lặp <code>while</code> .	Thêm lệnh <code>i += 1</code> vào thân lặp.	Có

Table 5: Mẫu bảng theo dõi lỗi cá nhân (Personal Error Log)

3. Chiến lược Sử dụng Trí tuệ Nhân tạo (AI) Hỗ trợ Học tập

3.1 Định vị vai trò chuẩn xác của AI trong học lập trình

Các mô hình ngôn ngữ lớn (ChatGPT, Claude, Gemini) là những trợ thủ học tập có sức mạnh chưa từng có. Tuy nhiên, giá trị giáo dục của AI phụ thuộc hoàn toàn vào **cách người học tương tác với nó**.

Mối nguy hại từ việc lạm dụng AI

Nếu sinh viên chỉ đơn giản gửi đề bài: *"Giải bài này hộ tôi"* rồi sao chép lời giải vào bài nộp:

- Sinh viên không hiểu thuật toán được xây dựng thế nào, tại sao lại dùng cấu trúc đó.
- Khi bước vào phòng thi không có AI, sinh viên hoàn toàn mất khả năng tự viết chương trình.
- Về lâu dài, kỹ năng tư duy phản biện và giải quyết vấn đề bị thoái hóa nghiêm trọng.

Định vị vai trò AI

Hãy coi AI là một **Gia sư Cá nhân kiêm Huấn luyện viên (AI Personal Tutor & Coach)**:

- Nhờ AI **giải thích** những khái niệm chưa hiểu theo góc nhìn đơn giản.
- Nhờ AI **gợi ý từng bước** khi bế tắc, thay vì xin trọn vẹn đáp án.
- Nhờ AI **đặt câu hỏi kiểm tra** để thẩm định độ hiểu bài của bản thân.
- Nhờ AI **review và tối ưu code** do chính mình tự tay viết ra.

3.2 Khung cấu trúc Prompt 4 thành phần

Để AI đưa ra câu trả lời phục vụ học tập tối ưu thay vì "làm bài hộ", một câu lệnh (prompt) hiệu quả cần hội đủ 4 yếu tố:

1. Ngữ cảnh (Context)	Bạn là ai, đang học môn gì, ở cấp độ nào?	<i>"Tôi là sinh viên năm nhất đang bắt đầu học môn Lập trình Python cơ bản..."</i>
2. Khó khăn (Problem)	Bạn đang vướng mắc chính xác ở điểm nào?	<i>"Tôi chưa phân biệt được sự khác nhau giữa biến số và giá trị gán cho biến..."</i>
3. Nhiệm vụ (Task)	Bạn muốn AI làm gì cụ thể?	<i>"Hãy giải thích bằng ví dụ đời thường và dùng kiến thức nâng cao..."</i>
4. Giới hạn (Constraint)	Ràng buộc cách AI phản hồi?	<i>"Sau khi giải thích, hãy đặt cho tôi 3 câu hỏi ngắn để tự kiểm tra, chưa vội đưa đáp án."</i>

3.3 Nguyên tắc 5T khi học cùng AI

Quy trình làm việc chuẩn mực giữa người học và AI được gói gọn trong nguyên tắc **5T**:

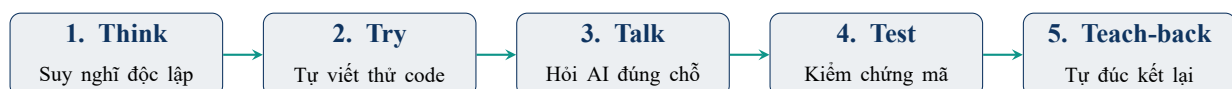


Figure 2: Mô hình 5T tương tác hiệu quả với AI trong học lập trình

- 1. Think (Suy nghĩ):** Đọc kỹ bài toán, xác định dữ liệu đầu vào (Input), đầu ra (Output), vạch ra thuật toán trước khi mở khung chat với AI.

2. **Try (Thử nghiệm):** Tự viết phiên bản code đầu tiên theo sức của mình, dù nó còn lỗi hoặc chưa hoàn thiện.
3. **Talk (Đổi thoại):** Chỉ hỏi AI về phần cụ thể đang vướng mắc; cung cấp mã nguồn hiện có, lỗi gặp phải và kỳ vọng.
4. **Test (Kiểm chứng):** Tuyệt đối không coi câu trả lời của AI là chân lý. Phải copy code vào IDE, chạy thử với nhiều bộ dữ liệu biên để kiểm tra.
5. **Teach-back (Giảng lại):** Đóng AI lại, tự diễn đạt lại cách giải quyết bằng lời của mình và ghi chú vào sổ tay.

3.4 Hệ thống gợi ý 3 cấp độ (Three-Level Hint System)

Khi gặp một bài tập học búa, hãy huấn luyện cho AI đưa ra gợi ý theo 3 nấc để duy trì quyền làm chủ tư duy của bạn:

Cơ chế gợi ý 3 nấc

- **Gợi ý Cấp 1 (Gợi ý ý tưởng – Idea Hint):** AI chỉ phân tích bài toán, nêu hướng tiếp cận logic, không đưa ra bất kỳ dòng code nào.
- **Gợi ý Cấp 2 (Gợi ý cấu trúc – Structure Hint):** AI gợi ý cấu trúc lệnh nên dùng (ví dụ: cần một vòng lặp for kết hợp với câu lệnh rẽ nhánh if, sử dụng toán tử chia lấy dư %).
- **Gợi ý Cấp 3 (Khung xương code – Scaffold Hint):** AI đưa ra khung sườn chương trình có chỗ trống (TODO) để sinh viên tự điền logic cốt lõi.

3.5 Bộ công cụ Prompt thực chiến (Prompt Toolkit)

Dưới đây là các mẫu câu lệnh chất lượng cao mà sinh viên có thể sao chép và áp dụng ngay trong quá trình tự học:

Mẫu Prompt thực hành: Giải thích một khái niệm lập trình mới

Tôi là người mới bắt đầu học lập trình Python. Hãy giải thích cho tôi khái niệm [TÊN KHÁI NIỆM, VD: Hàm đệ quy].

Yêu cầu:

1. Giải thích bản chất bằng ngôn ngữ dễ hiểu kèm 1 ví dụ tương đồng trong đời sống thực tế.
2. Đưa ra 1 ví dụ code Python ngắn gọn (dưới 10 dòng) minh họa.
3. Chỉ ra 1 lỗi phổ biến mà người mới rất hay mắc phải.
4. Đặt cho tôi 2 câu hỏi kiểm tra độ hiểu, chưa đưa ra đáp án ngay.

Mẫu Prompt thực hành: Hỏi gợi ý giải bài tập (không xin đáp án trọn gói)

Tôi đang tự làm bài tập sau: “[NỘI DUNG ĐỀ BÀI]”

Ý tưởng và đoạn code tôi đã thử viết như sau:

[DÁN CODE CỦA BẠN VÀO ĐÂY]

Hiện tại tôi đang bị tắc ở bước [MÔ TẢ KHÓ KHĂN].

Yêu cầu: TUYỆT ĐỐI KHÔNG viết toàn bộ code giải bài. Hãy cho tôi GỢI Ý CẤP ĐỘ 1 (gợi ý về mặt thuật toán/ý tưởng) để tôi có thể tự suy nghĩ và viết tiếp.

Mẫu Prompt thực hành: Hỗ trợ tìm lỗi và Debugging

Tôi viết đoạn code Python sau để giải quyết yêu cầu: [MÔ TẢ MỤC TIÊU CỦA CHƯƠNG TRÌNH].
Code hiện tại của tôi:
[DÁN CODE CỦA BẠN VÀO ĐÂY]
Khi chạy chương trình, tôi nhận được thông báo lỗi: [DÁN THÔNG BÁO LỖI HOẶC KẾT QUẢ SAI].
Yêu cầu:
1. Giải thích nguyên nhân tại sao lỗi này xuất hiện.
2. Chỉ rõ dòng code nào đang tiềm ẩn vấn đề.
3. Đặt cho tôi một câu hỏi gợi mở để tôi tự mình tìm cách sửa lỗi, KHÔNG sửa lại toàn bộ code của tôi.

Mẫu Prompt thực hành: Đánh giá và Tối ưu hóa mã nguồn (Code Review)

Tôi đã hoàn thành xong bài tập lập trình này và chương trình đã chạy ra kết quả đúng:
[DÁN CODE HOÀN CHỈNH VÀO ĐÂY]
Hãy đóng vai trò là một chuyên gia lập trình và Giảng viên Python:
1. Đánh giá chất lượng mã nguồn: cách đặt tên biến, tính rõ ràng, cấu trúc logic.
2. Mã nguồn này có xử lý được các trường hợp biên (edge cases) không?
3. Hãy gợi ý 1 điểm có thể viết gọn gàng hoặc tối ưu hơn theo phong cách chuẩn Python (Pythonic), giải thích rõ lý do.

Mẫu Prompt thực hành: Tạo Quiz tự kiểm tra kiến thức

Tôi vừa học xong các chủ đề: [DANH SÁCH CHỦ ĐỀ, VD: List, Tuple và Slicing].
Hãy tạo cho tôi một bài kiểm tra ngắn gồm 6 câu hỏi:
- 2 câu trắc nghiệm 4 lựa chọn A, B, C, D.
- 2 câu Dự đoán kết quả (Predict the Output).
- 2 câu Tìm lỗi sai trong đoạn code (Find the Bug).
Lưu ý: Chỉ sử dụng kiến thức ở mức cơ bản, KHÔNG hiển thị đáp án trước. Hãy chờ tôi gửi câu trả lời rồi mới chấm điểm và giải thích chi tiết.

3.6 Bảng kiểm thẩm định thông tin do AI sinh ra (Verification Checklist)

Trước khi chấp nhận bất kỳ thông tin nào do AI cung cấp, sinh viên bắt buộc phải tự đối chiếu với bảng kiểm tra 7 điểm sau:

- ☐ **1. Tính tương thích chương trình:** Đoạn code AI đưa ra có dùng các thư viện ngoài hoặc kiến thức nâng cao vượt quá phạm vi bài học hiện tại hay không?
- ☐ **2. Chạy thử nghiệm thực tế:** Toàn bộ code mẫu đã được copy vào IDE và chạy thử không có lỗi cú pháp chưa?
- ☐ **3. Khớp kết quả kỳ vọng:** Kết quả in ra trên terminal có đúng từng chữ so với yêu cầu của đề bài chưa?
- ☐ **4. Kiểm tra trường hợp biên:** Code có chạy đúng khi người dùng nhập số 0, số âm, chuỗi rỗng hoặc danh sách rỗng không?
- ☐ **5. Chuẩn xác về mặt thuật ngữ:** Thuật ngữ AI giải thích có đồng nhất với giáo trình chính thức của học phần (*Python for Everyone*) không?
- ☐ **6. Tự giải thích từng dòng:** Bạn có thể chỉ tay vào từng dòng lệnh trong lời giải của AI và giải thích cặn kẽ tại sao nó lại ở đó không?
- ☐ **7. Tái tạo độc lập:** Nếu xóa đoạn chat AI đi, bạn có thể tự mình viết lại giải pháp đó trong vòng 10 phút không?

Quy tắc vàng bất di bất dịch

“TUYỆT ĐỐI KHÔNG BAO GIỜ NỘP HOẶC LƯU LẠI MÃ NGUỒN MÀ BẢN THÂN BẠN KHÔNG THỂ TỰ MÌNH GIẢI THÍCH TƯỜNG TẬN.”

4. Kế hoạch Hành động Khởi đầu Học phần

Để đạt kết quả học tập xuất sắc (Điểm A/A+) trong môn học **DSAI1003**, sinh viên hãy bắt đầu ngay trong tuần đầu tiên bằng các hành động cụ thể sau:

1. Chuẩn bị môi trường máy tính:

- Cài đặt phiên bản Python mới nhất (Python 3.11+) từ trang chủ <https://www.python.org/>.
- Cài đặt trình soạn thảo mã nguồn **Visual Studio Code (VS Code)** cùng tiện ích mở rộng *Python Extension Pack*.
- Khởi tạo tài khoản **Google Colab** để sẵn sàng thực hành mọi lúc mọi nơi ngay trên trình duyệt web.

2. Tải và lưu trữ tài liệu học tập:

- Bookmark kho lưu trữ học liệu chính thức trên GitHub: <https://github.com/vudmvn/Fundamental-of-Programming-Concepts-in-Python>.
- Đọc trước Đề cương chi tiết (syllabus-vn.md) và Chương 1 giáo trình *Python for Everyone*.

3. Thiết lập hệ thống ghi chép học tập cá nhân:

- Khởi tạo thư mục học tập `DSAI1003_Python/` trên máy tính hoặc Google Drive.
- Tạo tệp `learning_log.md` để ghi chép bài học, lỗi sai và các bài học kinh nghiệm sau mỗi tuần.
- Chuẩn bị sẵn bộ khung Prompt Toolkit trên ứng dụng ghi chú để sử dụng tương tác cùng AI một cách văn minh, hiệu quả.

**CHÚC CÁC BẠN SINH VIÊN KHOA KH DỮ LIỆU & TRÍ TUỆ NHÂN TẠO
CÓ MỘT KỲ HỌC TRUYỀN CẢM HỨNG, LÀM CHỦ TƯ DUY LẬP TRÌNH
& TẬN DỤNG ĐỈNH CAO CÔNG NGHỆ AI ĐỂ BỨC PHÁ NĂNG LỰC CÁ NHÂN!**

**Khoa Khoa học Dữ liệu & Trí tuệ Nhân tạo – Trường Công nghệ
Đại học Kinh tế Quốc dân (NEU)
Hà Nội, Năm học 2025 – 2026**